SPECIFYING FILES BY NAME

OBJECTIVES

After completing this section, you should be able to specify the location of files relative to the current working directory and by absolute location, determine and change the working directory, and list the contents of directories.

ABSOLUTE PATHS AND RELATIVE PATHS

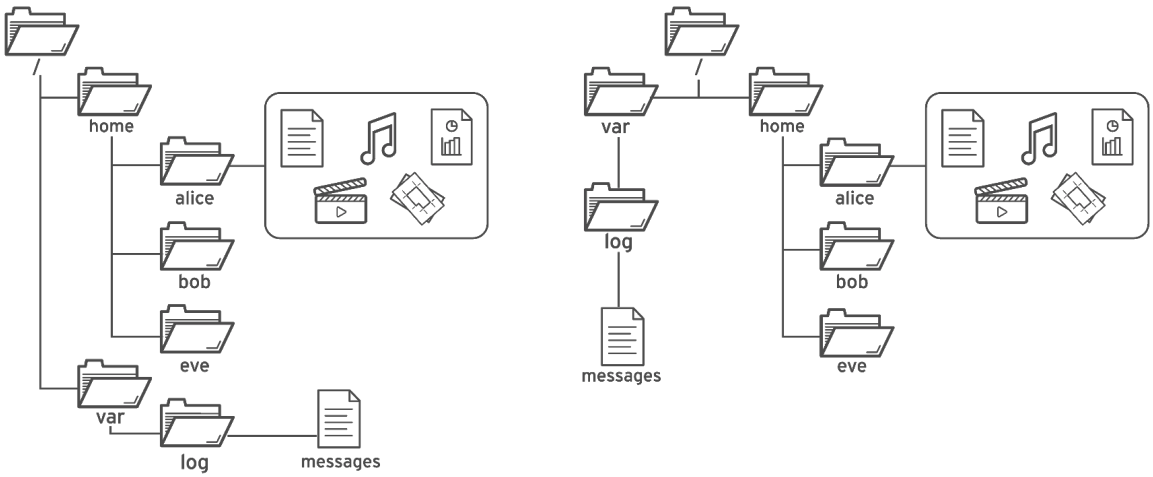

Figure 3.2: The common file browser view (left) is equivalent to the top-down view (right).

The *path* of a file or directory specifies its unique file system location. Following a file path traverses one or more named subdirectories, delimited by a forward slash (/), until the destination is reached. Directories, also called *folders*, contain other files and other subdirectories. They can be referenced in the same manner as files.

IMPORTANT

A space character is acceptable as part of a Linux file name. However, spaces are also used by the shell to separate options and arguments on the command line. If you enter a command that includes a file that has a space in its name, the shell can misinterpret the command and assume that you want to start a new file name or other argument at the space.

It is possible to avoid this by putting file names in quotes. However, if you do not need to use spaces in file names, it can be simpler to simply avoid using them.

Absolute Paths

An *absolute path* is a *fully qualified* name, specifying the files exact location in the file system hierarchy. It begins at the root (**/**) directory and specifies each subdirectory that must be traversed to reach the specific file. Every file in a file system has a unique absolute path name, recognized with a simple rule: A path name with a forward slash (**/**) as the first character is an absolute path name. For example, the absolute path name for the system message log file is **`/var/log/`**

messages. Absolute path names can be long to type, so files may also be located *relative* to the current working directory for your shell prompt.

The Current Working Directory and Relative Paths

When a user logs in and opens a command window, the initial location is normally the user's home directory. System processes also have an initial directory. Users and processes navigate to other directories as needed; the terms *working directory* or *current working directory* refer to their *current* location.

Like an absolute path, a *relative path* identifies a unique file, specifying only the path necessary to reach the file from the working directory. Recognizing relative path names follows a simple rule: A path name with *anything other than* a forward slash as the first character is a relative path name. A user in the **/var** directory could refer to the message log file relatively as **log/messages**.

Linux file systems, including, but not limited to, ext4, XFS, GFS2, and GlusterFS, are case-sensitive. Creating **FileCase.txt** and **filecase.txt** in the same directory results in two unique files.

Non-Linux file systems might work differently. For example, VFAT, Microsoft's NTFS, and Apple's HFS+ have *case preserving* behavior. Although these file systems are *not* case-sensitive, they do display file names with the original capitalization used when the file was created. Therefore, if you tried to make the files in the preceding example on a VFAT file system, both names would be treated as pointing to the same file instead of two different files.

NAVIGATING PATHS

The **pwd** command displays the full path name of the current working directory for that shell. This can help you determine the syntax to reach files using relative path names. The **ls** command lists directory contents for the specified directory or, if no directory is given, for the current working directory.

```
[user@host ~]$ pwd
/home/user
[user@host ~]$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
[user@host ~]$
```

Use the **cd** command to change your shell's current working directory. If you do not specify any arguments to the command, it will change to your home directory.

In the following example, a mixture of absolute and relative paths are used with the **cd** command to change the current working directory for the shell.

```
[user@host ~]$ pwd
/home/user
[user@host ~]$ cd Videos
[user@host Videos]$ pwd
/home/user/Videos
[user@host Videos]$ cd /home/user/Documents
[user@host Documents]$ pwd
/home/user/Documents
[user@host Documents]$ cd
[user@host ~]$ pwd
/home/user
```

```
[user@host ~]$
```

As you can see in the preceding example, the default shell prompt also displays the last component of the absolute path to the current working directory. For example, for **/home/user/Videos**, only **Videos** displays. The prompt displays the tilde character (~) when your current working directory is your home directory.

The **touch** command normally updates a file's timestamp to the current date and time without otherwise modifying it. This is useful for creating empty files, which can be used for practice, because "touching" a file name that does not exist causes the file to be created. In the following example, the **touch** command creates practice files in the **Documents** and **Videos** subdirectories.

```
[user@host ~]$ touch Videos/blockbuster1.ogg
[user@host ~]$ touch Videos/blockbuster2.ogg
[user@host ~]$ touch Documents/thesis_chapter1.odf
[user@host ~]$ touch Documents/thesis_chapter2.odf
[user@host ~]$
```

The **ls** command has multiple options for displaying attributes on files. The most common and useful are **-l** (long listing format), **-a** (all files, including *hidden* files), and **-R** (recursive, to include the contents of all subdirectories).

```
[user@host ~]$ ls -l
total 15
drwxr-xr-x.  2 user user 4096 Feb  7 14:02 Desktop
drwxr-xr-x.  2 user user 4096 Jan  9 15:00 Documents
drwxr-xr-x.  3 user user 4096 Jan  9 15:00 Downloads
drwxr-xr-x.  2 user user 4096 Jan  9 15:00 Music
drwxr-xr-x.  2 user user 4096 Jan  9 15:00 Pictures
drwxr-xr-x.  2 user user 4096 Jan  9 15:00 Public
drwxr-xr-x.  2 user user 4096 Jan  9 15:00 Templates
drwxr-xr-x.  2 user user 4096 Jan  9 15:00 Videos
[user@host ~]$ ls -la
total 15
drwx------. 16 user user   4096 Feb  8 16:15 .
drwxr-xr-x.  6 root root   4096 Feb  8 16:13 ..
-rw-------.  1 user user  22664 Feb  8 00:37 .bash_history
-rw-r--r--.  1 user user     18 Jul  9  2013 .bash_logout
-rw-r--r--.  1 user user    176 Jul  9  2013 .bash_profile
-rw-r--r--.  1 user user    124 Jul  9  2013 .bashrc
drwxr-xr-x.  4 user user   4096 Jan 20 14:02 .cache
drwxr-xr-x.  8 user user   4096 Feb  5 11:45 .config
drwxr-xr-x.  2 user user   4096 Feb  7 14:02 Desktop
drwxr-xr-x.  2 user user   4096 Jan  9 15:00 Documents
drwxr-xr-x.  3 user user   4096 Jan 25 20:48 Downloads
drwxr-xr-x. 11 user user   4096 Feb  6 13:07 .gnome2
drwx------.  2 user user   4096 Jan 20 14:02 .gnome2_private
-rw-------.  1 user user  15190 Feb  8 09:49 .ICEauthority
drwxr-xr-x.  3 user user   4096 Jan  9 15:00 .local
drwxr-xr-x.  2 user user   4096 Jan  9 15:00 Music
drwxr-xr-x.  2 user user   4096 Jan  9 15:00 Pictures
drwxr-xr-x.  2 user user   4096 Jan  9 15:00 Public
drwxr-xr-x.  2 user user   4096 Jan  9 15:00 Templates
drwxr-xr-x.  2 user user   4096 Jan  9 15:00 Videos
```

```
[user@host ~]$
```

The two special directories at the top of the listing refer to the current directory (**.**) and the *parent* directory (**..**). These special directories exist in every directory on the system. You will discover their usefulness when you start using file management commands.

IMPORTANT

File names beginning with a dot (.) indicate *hidden* files; you cannot see them in the normal view using **ls** and other commands. This is *not* a security feature. Hidden files keep necessary user configuration files from cluttering home directories. Many commands process hidden files only with specific command-line options, preventing one user's configuration from being accidentally copied to other directories or users.

To protect file *contents* from improper viewing requires the use of *file permissions*.

```
[user@host ~]$ ls -R
.:
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos

./Desktop:

./Documents:
thesis_chapter1.odf  thesis_chapter2.odf

./Downloads:

./Music:

./Pictures:

./Public:

./Templates:

./Videos:
blockbuster1.ogg  blockbuster2.ogg
[user@host ~]$
```

The **cd** command has many options. A few are so useful as to be worth practicing early and using often. The command **cd -** changes to the previous directory; where the user was *previously* to the current directory. The following example illustrates this behavior, alternating between two directories, which is useful when processing a series of similar tasks.

```
[user@host ~]$ cd Videos
[user@host Videos]$ pwd
/home/user/Videos
[user@host Videos]$ cd /home/user/Documents
[user@host Documents]$ pwd
/home/user/Documents
[user@host Documents]$ cd -
[user@host Videos]$ pwd
```

```
/home/user/Videos
[user@host Videos]$ cd -
[user@host Documents]$ pwd
/home/user/Documents
[user@host Documents]$ cd -
[user@host Videos]$ pwd
/home/user/Videos
[user@host Videos]$ cd
[user@host ~]$
```

The **cd ..** command uses the **..** hidden directory to move up one level to the *parent* directory, without needing to know the exact parent name. The other hidden directory (**.**) specifies the *current directory* on commands in which the current location is either the source or destination argument, avoiding the need to type out the directory's absolute path name.

```
[user@host Videos]$ pwd
/home/user/Videos
[user@host Videos]$ cd .
[user@host Videos]$ pwd
/home/user/Videos
[user@host Videos]$ cd ..
[user@host ~]$ pwd
/home/user
[user@host ~]$ cd ..
[user@host home]$ pwd
/home
[user@host home]$ cd ..
[user@host /]$ pwd
/
[user@host /]$ cd
[user@host ~]$ pwd
/home/user
[user@host ~]$
```

REFERENCES

info libc 'file name resolution' (*GNU C Library Reference Manual*)

- Section 11.2.2 File name resolution

bash(1), **cd**(1), **ls**(1), **pwd**(1), **unicode**(7), and **utf-8**(7) man pages

UTF-8 and Unicode
http://www.utf-8.com/